

to say that having knowledge of a user's password hash is useless, it isn't. It just so happens to be that computing the original password given the hash is not a trivial task.

An authentication system which uses password hashes would work as follows:

Registration:

- User creates an account using a particular password.
- The registration system stores the password hash, not the password in the database.

Authentication:

- User tries to log into the website with a password
- The application computes a hash of the password the user enters
- The application checks to see if this hash matches the saved hash for the password.
- If they match, the user is authenticated.

Let us take a very simple example of how this might work.

Let us take a security mechanism of an application that allows only 6-digit numeric passwords – such as those found in ATMs. This security system takes a user's password, and stores it as a hash of the original numeric password. That hashing function derives the hash of a number by taking the average of pairs of numbers in the original password.

So the password: 864159 [password]

Would become: 737 [hash]

Derived as:

$$\frac{8+6}{2} = 7 \quad \frac{4+1}{2} = 2.5 \simeq 3 \quad \frac{5+9}{2} = 7$$

Now, while it is very easy and fast to calculate the “hash” here, it is not possible to get back the password from this hash. However, as long as the user is inputting the same number, the resulting hash will be the same. Therefore testing hash of the number is as good as testing the original.

Since the “hash” is much smaller than the password, it is clear that many passwords will share the same hash. This kind of scenario is quite common with real hashing algorithms, which produce hashes of a fixed length. It is called a collision, and while developing a hashing algorithm, great care is taken to ensure that one cannot derive two inputs that produce the same hash easily. Even the smallest of changes in the input should alter the hash. Real-life hashing algorithms are significantly more complicated than the example described here of course!